数据结构与算法作业解析

数组地址计算与稀疏矩阵三元组

作业 5.1–5.2

从“会算”到“会辨析、会迁移”

原题 LaTeX 重建·超详细解析·易错点·举一反三
跨课程知识网络·国内外公开课程题对标·学科科普与技术史

C/C++ 中常用的**行优先（最右下标变化最快）**为主解，同时给出列优先情况下 生成日期:
的替代结果，避免因题面截取造成隐含条件歧义。

2026-09-23

目录

第一部分　全部原始作业题目 **4**

第二部分　全部答案与超详细解析 **5**

1　第 5.1 题：三维数组地址与存储空间 **5**

1.1　考查知识点 5
1.2　已知条件与各维长度 5
1.3　行优先地址公式从哪里来 5
1.4　第 (1) 问：$a_{1,6,8}$ 的起始地址 6
1.5　第 (2) 问：数组 A 的总存储空间 6
1.6　另一种写法：步长（stride）法 7
1.7　结果检验 7
1.8　如果采用列优先，会发生什么？ 7

2　第 5.2 题：稀疏矩阵三元组表示 **8**

2.1　考查知识点 8
2.2　逐行扫描非零元 8
2.3　标准三元组顺序表 8
2.4　程序中的 0-based 版本 9
2.5　结果检验：从三元组反向重建 9

第三部分　重点题拓展、举一反三与学科知识专题 **10**

3　本次作业知识体系总览 **10**

4　题目分类与筛选 **11**

5　重点题深度回顾：5.1 的“母题结构” **11**

5.1　母题的一般形式 11
5.2　看到什么信号，就该想到它 11

6 重点题举一反三：数组地址 **11**

6.1 变式 1：下界不再是 0 11

6.2 变式 2：反向由地址求下标 12

6.3 变式 3：同一数组改成列优先 12

7 重点题举一反三：稀疏矩阵 **13**

7.1 变式 1：从矩阵写三元组 13

7.2 变式 2：由三元组重建矩阵 13

7.3 变式 3：什么时候三元组才真正省空间? 13

8 易错题专题：错误机制与自检 **14**

9 相似概念辨析 **15**

10 核心知识系统回顾 **15**

10.1 数组为什么能 $O(1)$ 随机访问 15

10.2 高维数组与混合进位制 15

10.3 COO、CSR、CSC 的关系 15

11 关联学科知识网络与跨课程迁移 **16**

12 学科科普：为什么“存储表示”是数据结构的核心 **17**

13 相关技术史：从线性内存到稀疏矩阵交换格式 **17**

14 国内课程与教材对标（不冒充真题） **17**

15 国外高校公开课程题与考试题对标 **18**

15.1 公开题 1：MIT 6.087 Practical Programming in C, Problem Set 6 (2010) 18

15.2 公开题 2：UC Berkeley CS61B, Spring 2015 Midterm 18

15.3 公开题 3：Princeton COS 125, Summer 2024, Programming Assignment 4: Arrays 19

15.4 公开题 4：University of Oxford, C++ Training Task on Multidimensional Arrays 19

15.5 公开题 5：University of Cambridge, Part IA Computing Tutorial . . 19

15.6 公开题 6：CMU 15-121, Spring 2010, Homework 5 20

15.7 公开题 7：Cornell CS312, Spring 2005, Problem Set 2 Part 3 20
15.8 公开题 8：USC CSCI 103L, Fall 2026, Homework 1 Problem 6 . . . 20
15.9 跨校横向比较 21
15.10 高校题中反复出现的“母题” 21

16 考试迁移总结 **21**
16.1 本次必须掌握 21
16.2 做题时的识别流程 22

第四部分　参考资料与权威来源 **23**

最终核验清单 **24**

第一部分　全部原始作业题目

5.1　三维数组地址与存储空间

有三维数组

$$A[0:7,\,0:8,\,0:9],$$

数组的起始地址是 1000，每个元素长为 2，试给出下面结果：

(1) 元素 $a_{1,6,8}$ 的起始地址；
(2) 数组 A 所占用的存储空间。

5.2　稀疏矩阵的三元组表示

已知稀疏矩阵如下，试写出三元组表示：

$$M=\begin{bmatrix}15 & 0 & 0 & 0 & 16\\ 0 & 6 & 0 & 0 & 0\\ 0 & 0 & 8 & 0 & 0\\ 9 & 0 & 0 & 18 & 0\end{bmatrix}.$$

第二部分　全部答案与超详细解析

1　第 5.1 题：三维数组地址与存储空间

1.1　考查知识点

本题考查三件事：

1. 多维数组各维长度的计算；
2. 行优先存储时，多维下标到一维线性偏移量的映射；
3. 数组总存储量 = 元素总数 × 单个元素长度。

最重要的隐含条件

题面截图没有显式写“以行序为主序”或“列序为主序”。中国数据结构教材讲数组顺序存储时通常采用**行优先（row-major）**作为基本模型，C/C++ 真正的多维数组也按最右下标变化最快的顺序连续存放。因此本题主解按行优先计算。若教师课堂另行规定列优先，则第 (1) 问地址不同，本文在后面单独给出。

1.2　已知条件与各维长度

数组下标范围为

$$0 \le i \le 7, \qquad 0 \le j \le 8, \qquad 0 \le k \le 9.$$

因此三个维度的元素个数分别是

$$N_1 = 7 - 0 + 1 = 8, \qquad N_2 = 8 - 0 + 1 = 9, \qquad N_3 = 9 - 0 + 1 = 10.$$

单个元素长度记为 $L = 2$，基地址（即 $A[0][0][0]$ 的起始地址）为

$$B = 1000.$$

1.3　行优先地址公式从哪里来

行优先意味着：k 最先变化，j 次之，i 最慢。也就是

$$A[0][0][0], A[0][0][1], \ldots, A[0][0][9], A[0][1][0], \ldots$$

连续排入一维存储空间。

对 $A[i][j][k]$，在它前面共有：

- i 个完整的“第一维块”，每块有 $N_2 N_3 = 9 \times 10 = 90$ 个元素；
- 当前第一维块内，有 j 个完整的“第二维行”，每行有 $N_3 = 10$ 个元素；
- 当前行内，在 k 之前有 k 个元素。

所以线性偏移量（按“元素个数”计）为

$$\text{offset}(i,j,k) = iN_2N_3 + jN_3 + k.$$

若各维下界不是 0，则一般式为

$$\text{offset} = (i-l_1)N_2N_3 + (j-l_2)N_3 + (k-l_3).$$

最终地址为

$$\text{LOC}(A[i][j][k]) = B + L \cdot \text{offset}.$$

1.4 第 (1) 问：$a_{1,6,8}$ 的起始地址

代入 $i=1, j=6, k=8$：

$$\begin{aligned}\text{offset}(1,6,8) &= 1\times 9\times 10 + 6\times 10 + 8\\ &= 90 + 60 + 8\\ &= 158.\end{aligned}$$

这表示 $A[1][6][8]$ 前面正好有 158 个元素。每个元素长 2，所以地址偏移为

$$158 \times 2 = 316.$$

因此

$$\text{LOC}(A[1][6][8]) = 1000 + 316 = 1316.$$

$$\boxed{\text{LOC}(a_{1,6,8}) = 1316}$$

1.5 第 (2) 问：数组 A 的总存储空间

元素总数为

$$8 \times 9 \times 10 = 720.$$

每个元素长为 2，因此总存储量为

$$720 \times 2 = 1440.$$

$$\boxed{S_A = 1440 \text{ 个地址单元（若长度单位为字节，则为1440 B）}}$$

若按字节地址理解，数组第一个字节地址为 1000，总共占 1440 B，因此最后一个字节地址为

$$1000 + 1440 - 1 = 2439.$$

而最后一个元素 $A[7][8][9]$ 的**起始地址**为

$$1000 + (720-1)\times 2 = 2438.$$

这两个数不同，是因为最后一个元素本身还占两个字节。

1.6 另一种写法：步长（stride）法

定义每增加一个下标单位，线性偏移增加多少元素：

$$S_k = 1, \qquad S_j = N_3 = 10, \qquad S_i = N_2 N_3 = 90.$$

于是

$$\text{offset} = iS_i + jS_j + kS_k = 1 \times 90 + 6 \times 10 + 8 = 158.$$

步长法和上面的“数前面有多少元素”完全等价，但更适合推广到四维、五维数组，也更接近 NumPy、张量库和编译器中对 strides 的描述。

1.7 结果检验

1. 边界检查：$1 \in [0,7]$、$6 \in [0,8]$、$8 \in [0,9]$，下标合法。
2. 偏移检查：$0 \le 158 < 720$，确实落在数组范围内。
3. 地址检查：1316 落在数组占用的地址区间 $[1000, 2439]$ 内。
4. 对齐直觉：元素长度为 2，所有元素起始地址都应与 1000 同奇偶；1316 为偶数，吻合。

易错点

易错 1：把下标最大值 7、8、9 误当成元素个数。正确长度是“上界 $-$ 下界 $+1$”，所以是 8、9、10。

易错 2：写成 $1 \times 8 \times 9 + 6 \times 9 + 8$。行优先时，$i$ 后面还剩第 2、3 维，因此 i 的步长是 9×10，不是 8×9。

易错 3：忘记最后乘元素长度 2。158 是**元素偏移量**，不是地址偏移量。

易错 4：混淆“最后一个元素起始地址”与“数组最后一个字节地址”。

1.8 如果采用列优先，会发生什么？

列优先（最左下标变化最快）时，线性偏移为

$$\text{offset}_{\text{col}} = i + jN_1 + kN_1N_2.$$

因此

$$1 + 6 \times 8 + 8 \times 8 \times 9 = 625,$$

地址为

$$1000 + 625 \times 2 = 2250.$$

所以：

行优先：1316	列优先：2250

总存储空间始终是 1440，不受行/列优先影响。若课堂明确规定某一种次序，应严格按该规定作答。

2 第 5.2 题：稀疏矩阵三元组表示

2.1 考查知识点

稀疏矩阵中大量元素为 0。若仍逐个存储所有零，空间会浪费。三元组顺序表只记录每个非零元的

$$(\text{行号}, \text{列号}, \text{元素值}).$$

教材通常还会额外保存矩阵的行数 μ、列数 ν 和非零元个数 t，从而完整确定矩阵规模。

2.2 逐行扫描非零元

原矩阵为

$$\begin{bmatrix} 15 & 0 & 0 & 0 & 16 \\ 0 & 6 & 0 & 0 & 0 \\ 0 & 0 & 8 & 0 & 0 \\ 9 & 0 & 0 & 18 & 0 \end{bmatrix}.$$

按**教材常用的 1-based 行列号**逐行扫描：

- 第 1 行：$(1,1,15)$，$(1,5,16)$；
- 第 2 行：$(2,2,6)$；
- 第 3 行：$(3,3,8)$；
- 第 4 行：$(4,1,9)$，$(4,4,18)$。

因此非零元个数为

$$t = 6.$$

矩阵规模是 4×5。

2.3 标准三元组顺序表

序号	行号 i	列号 j	值 e
1	1	1	15
2	1	5	16
3	2	2	6
4	3	3	8
5	4	1	9
6	4	4	18

即

$$\boxed{\{(1,1,15),(1,5,16),(2,2,6),(3,3,8),(4,1,9),(4,4,18)\}}$$

若采用带元数据的三元组表头，可写

$$\boxed{(\mu,\nu,t)=(4,5,6)}.$$

2.4 程序中的 0-based 版本

若用 C/C++ 数组下标习惯，从 0 开始，则同一个矩阵的非零元坐标应整体减 1：

$$(0,0,15),(0,4,16),(1,1,6),(2,2,8),(3,0,9),(3,3,18).$$

这两种写法都能表示同一个数学矩阵，但**绝不能在同一张表中混用 1-based 和 0-based**。

2.5 结果检验：从三元组反向重建

把 6 个三元组逐个放回 4×5 的零矩阵：

$$(1,1)\leftarrow15,\ (1,5)\leftarrow16,\ (2,2)\leftarrow6,\ (3,3)\leftarrow8,\ (4,1)\leftarrow9,\ (4,4)\leftarrow18,$$

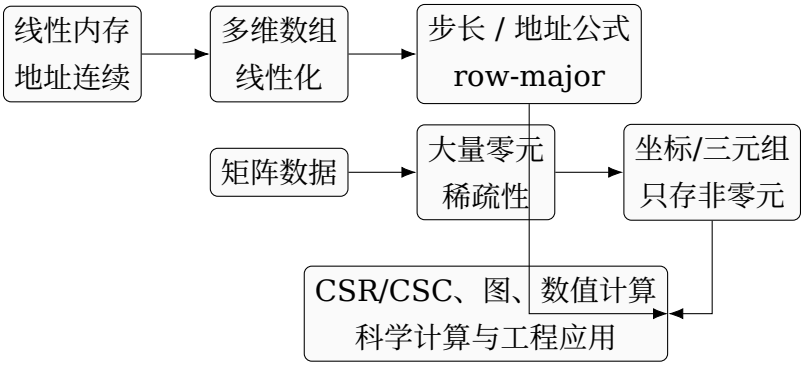
其余位置保持 0，恰好得到原矩阵，因此表示正确。

易错点

易错 1：把 0 也写入三元组。三元组压缩的目的就是只存非零元。
易错 2：漏掉矩阵尺寸。单独给出三元组集合时，若没有 μ,ν，尾部全零的行或列可能无法从数据本身推断。
易错 3：行列号顺序写反。约定一般是 (i,j,e)，不是 (j,i,e)。
易错 4：转成程序实现时仍沿用 1-based 坐标，却又拿它直接访问 0-based 数组。

第三部分　重点题拓展、举一反三与学科知识专题

3 本次作业知识体系总览



知识点	核心概念	典型公式/表示	对应原题	常见用途
多维数组顺序存储	多维下标映射为一维偏移	$((iN_2)+j)N_3+k$	5.1	数组、图像、张量、网格
地址计算	基地址 + 偏移 × 元素长度	$B+L\cdot\text{offset}$	5.1	指针、编译器、内存布局
步长 stride	每维下标加 1 造成的线性偏移	$(N_2N_3, N_3, 1)$	5.1	NumPy/张量、缓存优化
稀疏矩阵	非零元远少于总元素	$\text{nnz}(A)\ll mn$	5.2	图、PDE、优化、机器学习
三元组/COO	存 (i,j,a_{ij})	三列：row,col,value	5.2	构造、交换格式、装配
压缩格式	去掉显式零元	COO/CSR/CSC	5.2 拓展	高性能稀疏线性代数

4 题目分类与筛选

题号	分类	原因
5.1	A 类重点核心题;C 类易错题	地址公式是数组顺序存储的母题；维长、步长、元素长度三处都容易错。
5.2	A 类重点核心题；B 类基础题;C 类易错题	三元组是稀疏矩阵最直观压缩表示；坐标基准、顺序和元数据容易混淆。

5 重点题深度回顾：5.1 的“母题结构”

5.1 母题的一般形式

给定 d 维数组

$$A[l_1 : u_1, \ldots, l_d : u_d],$$

基地址 B、元素长度 L，求 $A[i_1, \ldots, i_d]$ 的地址。首先计算

$$N_r = u_r - l_r + 1.$$

行优先下第 r 维的步长为

$$S_r = \prod_{q=r+1}^{d} N_q,$$

最右一维 $S_d = 1$。所以

$$\mathrm{LOC} = B + L \sum_{r=1}^{d} (i_r - l_r) S_r.$$

5.2 看到什么信号，就该想到它

只要题目同时出现“多维数组 + 基地址 + 每元素长度 + 某个下标”，就应立即联想到:

$$\text{先确定存储次序} \to \text{算各维长度} \to \text{算 stride} \to \text{线性偏移} \to \text{乘元素长度}.$$

6 重点题举一反三：数组地址

6.1 变式 1：下界不再是 0

题目： 三维数组 $B[-2 : 3, 1 : 5, 4 : 10]$ 按行优先连续存储，基地址为 2000，每个元素 4 B。求 $B[1, 3, 9]$ 的起始地址与整个数组空间。

解： 各维长度

$$N_1 = 6, \quad N_2 = 5, \quad N_3 = 7.$$

偏移量

$$\begin{aligned}&(1-(-2))\times 5\times 7+(3-1)\times 7+(9-4)\\&=3\times 35+2\times 7+5=124.\end{aligned}$$

地址

$$2000+124\times 4=2496.$$

总空间

$$6\times 5\times 7\times 4=840\ \text{B}.$$

$$\boxed{\text{LOC}(B[1,3,9])=2496,\qquad S_B=840\ \text{B}}$$

与原题相比改变了什么：增加了非零下界，因此每个下标必须先减对应下界，不能直接把 i,j,k 代入。

6.2 变式 2：反向由地址求下标

题目： $C[0:3,0:4,0:5]$ 按行优先存储，基地址 500，每个元素 4 B。某元素起始地址为 812，求其三维下标。

解：先由地址得到元素偏移

$$q=\frac{812-500}{4}=78.$$

第三维长度 $N_3=6$，第二维长度 $N_2=5$。做连续的“商与余数”分解：

$$k=78\bmod 6=0,\qquad \left\lfloor\frac{78}{6}\right\rfloor=13.$$

再分解 13：

$$j=13\bmod 5=3,\qquad i=\left\lfloor\frac{13}{5}\right\rfloor=2.$$

$$\boxed{C[2,3,0]}$$

检验：$2\times 5\times 6+3\times 6+0=78$。

6.3 变式 3：同一数组改成列优先

对原题的 $A[1,6,8]$，若改成列优先：

$$\text{offset}=1+6\times 8+8\times 8\times 9=625,$$

故地址为 2250。这个变式直接检验你是否真正理解“存储次序决定哪一维变化最快”。

7 重点题举一反三：稀疏矩阵

7.1 变式 1：从矩阵写三元组

给定

$$P=\begin{bmatrix}0&4&0&0&0&-2\\0&0&0&5&0&0\\7&0&0&0&0&0\\0&0&9&0&3&0\\0&0&0&0&0&6\end{bmatrix}.$$

按 1-based 行优先三元组为

$$\boxed{(1,2,4),(1,6,-2),(2,4,5),(3,1,7),(4,3,9),(4,5,3),(5,6,6)}.$$

元数据为 $(\mu,\nu,t)=(5,6,7)$。

7.2 变式 2：由三元组重建矩阵

已知 Q 为 4×4 矩阵，三元组

$$(1,1,2),(2,4,-1),(3,2,5),(4,4,8).$$

其余位置均为 0，因此

$$Q=\begin{bmatrix}2&0&0&0\\0&0&0&-1\\0&5&0&0\\0&0&0&8\end{bmatrix}.$$

这个逆过程是检查三元组是否漏项、错位的最快方法之一。

7.3 变式 3：什么时候三元组才真正省空间?

假设一个 1000×1000 的整型矩阵，密集存储每个元素占 4 B；简化的 COO 三元组中，行号、列号、数值都各占 4 B，即每个非零元约 12 B。

密集存储需要

$$1000\times1000\times4=4{,}000{,}000\ \text{B}.$$

若非零元为 k，COO 约需 $12k$ B。要比密集格式省空间：

$$12k<4{,}000{,}000\quad\Longrightarrow\quad k<333{,}333.3.$$

即密度约低于 33.3% 时，单从这个粗略模型看就开始节省空间。若希望至少节省 90%：

$$12k\le400{,}000\quad\Longrightarrow\quad k\le33{,}333,$$

密度约需不高于 3.33%。实际工程还要考虑结构体对齐、索引位宽、格式元数据与访问效率，因此阈值并非固定常数。

8 易错题专题：错误机制与自检

错误模式	为什么会错	正确处理	快速自检
把 $0:7$ 当 7 个元素	忘了上下界都包含	长度 $u-l+1$	小规模手数下标
步长写反	未判断哪一维变化最快	行优先最右维 stride=1	列出前 12 个元素
偏移忘乘元素长度	混淆“第几个元素”和“地址差”	地址 = 基址 + 元素偏移 $\times L$	看单位是否仍为“元素”
行列优先混用	把数学矩阵显示方式当成内存布局	先明确 layout 再写公式	比较相邻地址
三元组包含 0	没理解压缩目标	只记录非零元	数三元组数是否等于 nnz
行列号反写	坐标顺序记忆模糊	固定 $(row, col, value)$	重建原矩阵
0-based/1-based 混用	数学与程序习惯不同	整份表示只选一种基准	检查左上角坐标
漏矩阵尺寸	以为三元组本身总能推断形状	保存 μ, ν, t	想象末行/末列全 0

9 相似概念辨析

概念对	核心区别	常见误区	判断方法
行优先 vs 列优先	哪个下标变化最快	以为只是“打印方向”不同	看相邻元素地址
多维数组 vs 指针数组	真多维数组通常是一块连续存储; 指针数组可能各行分散	用同一个地址公式套所有 ‘T**‘	看对象类型与内存分配方式
密集矩阵 vs 稀疏矩阵	前者存全部元素, 后者利用大量 0 的结构	以为“有几个 0”就叫稀疏	比较 nnz 与 mn 及算法目标
三元组 COO vs CSR	COO 对每个非零元显式存行列；CSR 用行指针压缩行号	以为都只能做存储不能运算	看访问模式和操作复杂度
矩阵下标 vs 内存地址	下标是逻辑坐标, 地址是物理/虚拟线性位置	把下标直接当地址偏移	必须经过线性化和元素长度缩放

10 核心知识系统回顾

10.1 数组为什么能 $O(1)$ 随机访问

固定维数、固定布局的数组，其任意元素地址都能通过有限次加法和乘法直接算出，无需从头遍历。因此按 RAM 模型，索引访问是 $O(1)$。这一点本质上依赖于**规则、可计算的线性化映射**。

10.2 高维数组与混合进位制

行优先的下标映射与“混合进位制”非常相似。三维数组 $N_1 \times N_2 \times N_3$ 中，线性下标

$$q = iN_2N_3 + jN_3 + k$$

就像把 (i, j, k) 视为不同“进位基数”的数字。反向求下标正是不断做整除和取余。

10.3 COO、CSR、CSC 的关系

三元组本质上就是现代稀疏计算中的 COO（Coordinate）思想：对每个非零元保存坐标和值。CSR（Compressed Sparse Row）进一步把“每个非零元都重复保存行号”改成“每行只记录起止位置”，通常包括:

$$\texttt{values},\quad \texttt{col_index},\quad \texttt{row_ptr}.$$

CSC 则对列做同样压缩。COO 更适合增量构造和交换，CSR/CSC 更适合按行/列进行高效计算。

11 关联学科知识网络与跨课程迁移

当前知识	关联课程/学科	关联知识	类型	为什么有关
数组地址	C/C++ 程序设计	指针算术、‘sizeof‘	强关联	数组下标最终会被编译为地址计算；指针加 1 会按元素大小缩放。
数组步长	计算机组成	Cache locality	后续/应用	连续访问更容易命中缓存；错误遍历顺序可能明显降低性能。
多维数组	图像处理	像素网格	应用	图像是二维/三维规则数组，行跨度与通道布局直接影响内存访问。
多维数组	机器学习	tensor stride	拓展	张量切片、转置常通过修改 shape/stride 实现，不一定复制数据。
稀疏矩阵	线性代数/数值分析	稀疏线性方程组	强关联	PDE 离散后常产生大型稀疏矩阵，存储与求解器必须利用稀疏性。
稀疏矩阵	图论	邻接矩阵	强关联	大图的邻接矩阵通常非常稀疏；非零元对应边。
稀疏矩阵	机器人/控制	大规模优化与图优化	拓展	SLAM、最小二乘、因子图中雅可比和正规方程常具有结构化稀疏性。

学习层级

当前课程必须掌握： 行优先地址计算、数组总空间、三元组表示。
建议复习： C/C++ 数组与指针、矩阵基本下标。
后续课程会系统学习： 缓存局部性、CSR/CSC、稀疏线性代数。
拓展了解即可： 张量 stride、图优化、GPU 稀疏算子。

12 学科科普：为什么“存储表示”是数据结构的核心

数据结构并不只是“链表、树、图的名称集合”。它真正研究的是：**如何把逻辑对象映射到机器可存取的表示，并让目标操作足够高效**。

数组与稀疏矩阵刚好展示了两种相反但互补的思路：

- 数组利用“规则性”：所有元素形状相同、位置规则，所以可以通过公式一步算出地址，换来 $O(1)$ 随机访问。
- 稀疏矩阵利用“冗余性”：大量零不值得存，所以牺牲部分直接随机访问便利，只保存有信息的非零元，换来巨大的空间节省。

工程设计往往就是在**空间、时间、访问模式、实现复杂度**之间做权衡。没有“永远最好的数据结构”，只有与操作需求匹配的表示。

13 相关技术史：从线性内存到稀疏矩阵交换格式

早期高级语言必须把数学上的多维结构映射到线性存储器。不同语言形成了不同约定：C/C++ 采用最右下标变化最快的行优先布局，而 Fortran 系传统上采用列优先。这个差异延续至今，并直接影响跨语言数值库接口与缓存访问顺序。

稀疏矩阵的发展同样受到内存和计算成本推动。大型科学与工程问题离散后会得到绝大多数元素为零的矩阵，显式存储全部零会浪费空间。因而形成了坐标表、按行/列压缩等多种格式。1996 年，NIST 发布 Matrix Market 交换格式的初始设计，其中的 coordinate format 直接用“坐标 + 数值”描述稀疏非零元，与本章三元组思想高度一致。Matrix Market 还将 Harwell-Boeing 等经典稀疏矩阵集合纳入公开测试生态，促进了数值算法的可复现实验与比较。

这段历史与本次作业的直接联系

你在 5.2 中写的 (i, j, a_{ij}) 并不是只存在于教材里的“教学格式”。现代科学计算中仍有非常接近的 COO / Matrix Market coordinate 表示。教材中的三元组，是理解现代稀疏线性代数数据格式的一座非常直接的桥。

14 国内课程与教材对标（不冒充真题）

公开检索中，可直接核验的中国高校“数组地址 + 三元组”官方考试原题较少。为遵守来源真实性，本节只做课程与教材对标，不把教材示例或课程提纲伪称为历年真题。

- 北京大学数学学院公开的《算法与数据结构》课程简介明确将“多维数组和稀疏矩阵”列入课程内容，说明本次两题正位于数据结构课程的经典数组/矩阵存储单元。
- 清华大学出版社公开教材样章《数组和稀疏矩阵》给出了三元组 (i, j, a_{ij})、按行序为主序排列以及 (m, n, len) 等结构化描述，可作为 5.2 的教材级交叉核验。

15 国外高校公开课程题与考试题对标

来源与版权说明

以下均给出可核验的高校公开页面。为避免大段复制课程材料，本文对题意做**学习性概述/同型化重述**，并给出独立解答；只有来源页面明确是考试时才标注“考试”。课程作业、教程与讲义均按其真实类型标注，不将其伪称为真题。

15.1 公开题 1：MIT 6.087 Practical Programming in C, Problem Set 6 (2010)

来源： MIT OpenCourseWare；6.087 Practical Programming in C；January IAP 2010；Problem Set 6；主题含 multidimensional arrays。
公开页面： https://ocw.mit.edu/courses/6-087-practical-programming-in-c-january-iap-2010/resources/problem-set-6/

同型迁移题：若 C 中有 ‘short A[8][9][10]‘,并假设 $\&A[0][0][0] = 1000$、‘sizeof(short)=2‘，求 $\&A[1][6][8]$。

解： C 真多维数组按行优先排列，偏移为 $1 \cdot 9 \cdot 10 + 6 \cdot 10 + 8 = 158$ 个 ‘short‘，因此地址

$$1000 + 158 \cdot 2 = 1316.$$

与原作业关系： 几乎同一母题，但把“数据结构地址公式”直接落到 C 的真实数组内存布局。

15.2 公开题 2：UC Berkeley CS61B, Spring 2015 Midterm

来源： UC Berkeley EECS HKN exam archive；CS61B Midterm, Spring 2015；其中一道题操作二维字符串数组并访问八邻域。
公开页面： https://hkn.eecs.berkeley.edu/examfiles/cs61b_sp15_mt1_sol.pdf

同型迁移题： 一个 5×6 二维数组最外一圈是固定边界，只更新内部元素。内部共有多少个单元？若每个内部单元都检查其 8 个邻居，一轮共做多少次邻居读取？

解： 内部行数 $5 - 2 = 3$，内部列数 $6 - 2 = 4$，所以内部单元数

$$3 \times 4 = 12.$$

每个检查 8 个邻居，总读取

$$12 \times 8 = 96.$$

与原作业关系： 从“地址映射”迁移到“二维数组边界与邻接访问”，训练下标范围意识。

15.3 公开题 3：Princeton COS 125, Summer 2024, Programming Assignment 4: Arrays

来源： Princeton University COS 125；Summer 2024；Programming Assignment 4: Arrays；含 perfect shuffle 的数组重排。
公开页面： https://www.cs.princeton.edu/~cos125/summer24/assignments/arrays/specification.php

同型迁移题： 长度为偶数 m 的数组做一次 perfect shuffle：旧数组前半段依次放到新数组偶数位置，后半段依次放到奇数位置。写出下标映射。

解： 对 $0 \le i < m/2$：

$$\text{old}[i] \to \text{new}[2i], \qquad \text{old}[i + m/2] \to \text{new}[2i + 1].$$

它本质上也是“逻辑下标 $\to$ 线性位置”的映射问题，只不过不是连续地址公式，而是置换。

15.4 公开题 4：University of Oxford, C++ Training Task on Multidimensional Arrays

来源： University of Oxford IT Learning；C++ comprehensive introduction；Task 2 要求观察多维数组的读写和下标。
公开页面： https://skills.it.ox.ac.uk/files/itlcppg001notescplusplusccomprehensiveintroductionpdf

同型迁移题： 对 ‘double ExamMarks[2][2]‘，按 C/C++ 行优先，若首地址为 B、‘double‘ 长 8 B，求 ‘ExamMarks[1][0]‘ 地址。

解： 偏移元素数为 $1 \times 2 + 0 = 2$，所以

$$\text{LOC} = B + 2 \times 8 = B + 16.$$

易错点： 二维可视化中的“第二行第一列”不等于列优先；C/C++ 仍按行连续。

15.5 公开题 5：University of Cambridge, Part IA Computing Tutorial

来源： University of Cambridge Engineering；Part IA Computing Course tutorial；公开材料含 multidimensional arrays 与 3×4、旋转矩阵示例。
公开页面： https://mi.eng.cam.ac.uk/~cipolla/resource/tutorial.pdf

同型迁移题： 设 ‘float M[3][4]‘，每个 ‘float‘ 4 B，首地址为 4000，求 ‘M[2][3]‘ 的地址。

解：

$$\text{offset} = 2 \times 4 + 3 = 11, \qquad \text{LOC} = 4000 + 11 \times 4 = 4044.$$

与原作业关系： 是 5.1 的二维降维版；如果二维公式不熟，三维就更容易写错。

15.6 公开题 6：CMU 15-121, Spring 2010, Homework 5

来源： Carnegie Mellon University；15-121 Spring 2010；Homework 5 Programming Project；用链式结构实现 sparse matrix，并包含 transpose。
公开页面： https://www.cs.cmu.edu/~tcortina/15-121sp10/homework5.html

同型迁移题： 将原作业 5.2 的矩阵转置，并写出转置矩阵的 1-based 三元组。

解： 转置就是每个三元组的行列互换，再按新行号排序：

$$\boxed{(1,1,15),(1,4,9),(2,2,6),(3,3,8),(4,4,18),(5,1,16)}.$$

新矩阵规模为 5×4，非零元数仍为 6。

15.7 公开题 7：Cornell CS312, Spring 2005, Problem Set 2 Part 3

来源： Cornell University；CS312 Spring 2005；Problem Set 2 Part 3: Sparse Matrices；用“每行的 (column,value) 列表”表示稀疏矩阵。
公开页面： https://www.cs.cornell.edu/courses/cs312/2005sp/hw/ps2/ps2.htm

同型迁移题： 把原矩阵改写成 Cornell 页面所示的 0-based“按行列表”思想：每一行只保留 $(\text{column},\text{value})$。

解：

$$\begin{aligned} R_0 &= [(0,15),(4,16)],\\ R_1 &= [(1,6)],\\ R_2 &= [(2,8)],\\ R_3 &= [(0,9),(3,18)]. \end{aligned}$$

矩阵列数为 5。这个表示比 COO 进一步利用了“当前列表已经隐含行号”。

15.8 公开题 8：USC CSCI 103L, Fall 2026, Homework 1 Problem 6

来源： University of Southern California；CSCI 103L Fall 2026；Homework 1 Problem 6: Sparse Matrix；以行/列双向链表只保存非零元。
公开页面： https://bytes.usc.edu/cs103/assignments/hw1.html

同型迁移题： 若稀疏矩阵某位置 (r,c) 当前为 0，把它设为非零值 v，为什么不能只保存 v？

解： 因为零元没有显式结点，若只保存 v 就无法知道它在原矩阵中的位置。至少必须保留坐标信息 (r,c) 与数值 v；若为了按行、按列高效遍历，还需要相应的链接指针或行/列索引结构。这正是三元组从“值”扩展为“行、列、值”三字段的根本原因。

15.9 跨校横向比较

学校/课程	公开材料类型	主要数组/稀疏主题	主要能力	与原作业关系
MIT 6.087	Problem Set	多维数组、指针	内存模型与编程	5.1 直接迁移
UC Berkeley CS61B	Midterm	2D 数组邻域	边界、索引、循环	5.1 下标意识
Princeton COS125	Assignment	数组重排	索引映射	5.1 映射思想
Oxford C++	Training Task	多维数组	读写与下标	5.1 二维基础
Cambridge IA	Tutorial	多维数组/矩阵	数组与工程计算	5.1 二维基础
CMU 15-121	Homework	sparse matrix/-transpose	数据结构设计	5.2 进阶
Cornell CS312	Problem Set	sparse row lists	表示不变量	5.2 格式迁移
USC CSCI103L	Homework	双向稀疏结构	动态内存/链结构	5.2 工程化

15.10 高校题中反复出现的“母题”

母题 A：规则数组的下标映射。识别信号是固定维度、固定布局、下标访问。核心是把多维坐标转换为线性位置。

母题 B：边界与邻域。识别信号是二维网格、周围元素、边界特殊处理。核心是合法下标范围与访问顺序。

母题 C：只保存“有信息的元素”。识别信号是大量 0、空间浪费、非零元操作。核心是“值 + 位置”以及维护表示不变量。

母题 D：表示决定操作复杂度。同样是稀疏矩阵，COO、行列表、双向链表、CSR 适合的操作不同，数据结构必须服务于目标操作。

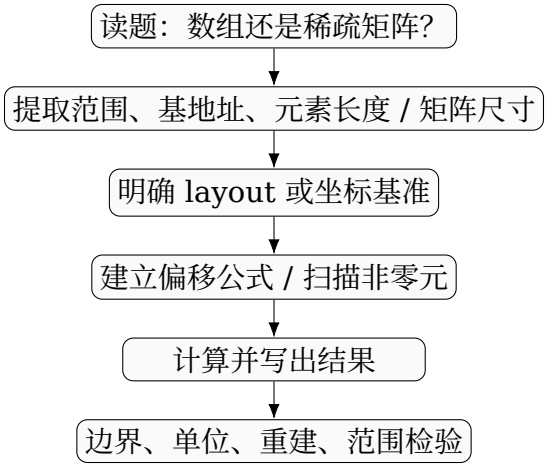
16 考试迁移总结

16.1 本次必须掌握

1. 维长度：$u-l+1$；
2. 行优先三维偏移：$(i-l_1)N_2N_3+(j-l_2)N_3+(k-l_3)$；
3. 地址：$B+L\cdot \text{offset}$；

4. 总空间：$N_1N_2\cdots N_d \times L$；
5. 三元组：$(row, col, value)$，并明确矩阵规模和非零元数；
6. 0-based 与 1-based 必须统一。

16.2 做题时的识别流程



一句话记忆，但不替代理解

数组地址：“先看谁变化最快，再算前面有多少元素，最后乘元素长度。”
三元组：“只存非零元，但不能只存值，还必须存它在哪里。”

第四部分　参考资料与权威来源

[1] 清华大学出版社公开教材样章，《算法与数据结构》第 5 章“数组和稀疏矩阵”，包含三元组顺序表及 $(row, col, data)$ 结构说明。https://www.tup.tsinghua.edu.cn/upload/books/yz/093572-01.pdf，访问：2026-09-23。

[2] 清华大学出版社公开教材样章，《数据结构知识点与习题精讲》相关章节，说明稀疏矩阵可按“行标、列标、元素值”三元组顺序表示。https://www.tup.tsinghua.edu.cn/upload/books/yz/093312-01.pdf，访问：2026-09-23。

[3] 北京大学数学学院，《算法与数据结构》课程简介，课程内容含“多维数组和稀疏矩阵”。https://www.math.pku.edu.cn/teachers/zhangnx/ds/intro.html，访问：2026-09-23。

[4] University of California, Irvine, ICS 46，“Multidimensional Data”，说明 C++ 多维数组的 row-major 连续布局。https://ics.uci.edu/~thornton/ics46/Notes/MultidimensionalData/，访问：2026-09-23。

[5] University of Oxford, C++ lecture notes / task on multidimensional arrays，说明多维数组下标与最右下标快速变化的存储方式。https://skills.it.ox.ac.uk/files/itlcppg001notescpluspluscomprehensiveintroductionpdf，访问：2026-09-23。

[6] University of Cambridge Engineering, Part IA Computing Course tutorial，含 multi-dimensional arrays 教学内容。https://mi.eng.cam.ac.uk/~cipolla/resource/tutorial.pdf，访问：2026-09-23。

[7] MIT OpenCourseWare, 6.087 Practical Programming in C, January IAP 2010, Problem Set 6；主题含 pointers to pointers、multidimensional arrays、stacks、queues。https://ocw.mit.edu/courses/6-087-practical-programming-in-c-january-iap-2010/resources/problem-set-6/，访问：2026-09-23。

[8] UC Berkeley EECS HKN Exam Archive, CS61B Midterm, Spring 2015，含二维数组邻域操作题。https://hkn.eecs.berkeley.edu/examfiles/cs61b_sp15_mt1_sol.pdf，访问：2026-09-23。

[9] Princeton University COS 125, Summer 2024, Programming Assignment 4: Arrays。https://www.cs.princeton.edu/~cos125/summer24/assignments/arrays/specification.php，访问：2026-09-23。

[10] Carnegie Mellon University, 15-121 Spring 2010, Homework 5, Sparse Matrix programming project。https://www.cs.cmu.edu/~tcortina/15-121sp10/homework5.html，访问：2026-09-23。

[11] Cornell University, CS312 Spring 2005, Problem Set 2, Part 3: Sparse Matrices。https://www.cs.cornell.edu/courses/cs312/2005sp/hw/ps2/ps2.htm，访问：2026-09-23。

[12] University of Southern California, CSCI 103L Fall 2026, Homework 1, Prob-

lem 6: Sparse Matrix。https://bytes.usc.edu/cs103/assignments/hw1.html，访问：2026-09-23。

[13] Per-Olof Persson et al., University of California, Berkeley, “Programming for Mathematical Applications: Sparse Matrices”，对比 dense、COO、CSC 等稀疏表示。https://persson.berkeley.edu/Programming_for_Mathematical_Applications/content/Sparse_Matrices/Matrix_Designs.html，访问：2026-09-23。

[14] NIST, R. Boisvert, R. Pozo, K. Remington, *The Matrix Market Exchange Formats: Initial Design*, NISTIR 5935, 1996。其 coordinate format 明确以坐标与数值描述稀疏非零元。https://www.nist.gov/publications/matrix-market-exchange-formats-initial-design，访问：2026-09-23。

[15] NIST Dictionary of Algorithms and Data Structures, “sparse matrix”，说明可用 coordinate format、lists、orthogonal lists 等压缩表示。https://xlinux.nist.gov/dads/HTML/sparsematrix.html，访问：2026-09-23。

最终核验清单

- 原题两题均已重建，且第一部分未提前插入答案；
- 5.1 主解在行优先假设下得到地址 1316、总空间 1440；列优先替代地址明确标为 2250；
- 5.2 非零元共 6 个，三元组逐一反向重建核验；
- 所有变式题已代回或重建检查；
- 高校公开材料均按真实类型标注，未把作业/教程伪称为考试；
- 中国高校官方真题若无法直接核验，则明确降级为“课程/教材对标”，不虚构学校、年份或题号。